Impact of Partial and Full Requirement Changes on Software Systems:

Research Analysis :

Paper A: Etezadi, R., Abualhaija, S., Arora, C., Briand, L. (2025/2026). *"LLM-Driven Cost-Effective Requirements Change Impact Analysis."* arXiv:2511.00262. (Accepted for ACM journal, 2026) Link: https://arxiv.org/abs/2511.00262

Paper B: Yildirim, S., Cevik, M., Parikh, D., Basar, A. (2023). *"Adaptive Fine-tuning for Multiclass Classification over Software Requirement Data."* arXiv:2301.00495. Link: https://arxiv.org/abs/2301.00495

Both papers are from 2020 onward as you asked, both are on arXiv (freely readable), and both are directly about software requirements and how they are classified/changed.

DEEP ANALYSIS OF EACH PAPER:

Point	PAPER A: "LLM-Driven Cost-Effective Requirements Change Impact Analysis"	PAPER B: "Adaptive Fine-tuning for Multiclass Classification over Software Requirement Data"
1 **Title**	LLM-Driven Cost-Effective Requirements Change Impact Analysis	Adaptive Fine-tuning for Multiclass Classification over Software Requirement Data
2 **Authors**	Romina Etezadi (U. Ottawa), Sallam Abualhaija (U. Luxembourg), Chetan Arora (Monash U.), Lionel Briand (U. Ottawa / U. Limerick)	Savas Yildirim, Mucahit Cevik, Devang Parikh (IBM), Ayse Basar (Toronto Metropolitan U.)
3 **Publication year**	2025 (posted arXiv Nov 2025; journal version 2026)	2023
4 **Research problem**	Engineers must manually find which other requirements are affected when one requirement changes ("inter-requirement Change Impact Analysis"). Manual work is slow, costly, error-prone — especially in regulated domains (satellite, medical, automotive).	Standard 2-stage fine-tuning (pretrain + fine-tune) assumes training data and real-world data share the same distribution, which is often false in real software task-management systems. Can extra "adaptive" pre-training fix this?

5 **Research objectives**	**Objective:** Build an automated LLM-based method that finds the "impact set" (list of affected requirements) for a given change, while keeping the number of requirements a human must check as small as possible.	Classify software requirement records (IBM DOORS tool) into **Type**, **Priority**, and **Severity**, using a 3-stage adaptive fine-tuning method, compared against strong baselines.
6 **Research questions**	RQ1: Which LLM is most stable across prompts/datasets? RQ2: Which prompt details matter most / best prompt? RQ3: Does Refinement+Filtering post-processing help? RQ4: How does ProReFiCIA compare to baselines? RQ5: Does adding domain knowledge (RAG) help?	Not formally numbered, but: (a) does adaptive fine-tuning beat plain 2-stage fine-tuning? (b) which of Type/Priority/Severity is hardest, and why?
7 Proposed methodology	Built 64 prompt versions using a Role-Instruction-Context-Examples (RICE) template; tested on 5 LLMs (GPT-4o, DeepSeek, LLaMA, Mistral, Gemini); added **Refinement** (re-checks skipped requirements) and **Filtering** (removes wrong guesses via an NLI model) steps — full pipeline called **ProReFiCIA**.	3-stage transfer learning: (1) start from public pretrained Transformer checkpoint (BERT/RoBERTa/DistilBERT); (2) continue unsupervised training (Masked Language Modeling) on ~80,000 unlabeled in-domain DOORS texts ("adaptive" step); (3) fine-tune the adapted model on labeled data for each of the 3 tasks.
8 **Dataset(s) used**	**WASP** (public, mobile-service-platform requirements); **SAT-DLink** (private, satellite system); **SAT-DLink-Test** (private, same satellite domain, unseen test set)	IBM DOORS Next Generation requirement records (task-management style tickets: bug/feature/story)
9 **Dataset size**	WASP: 72 requirements, 5 change requests, 22 impacted. SAT-DLink: 192 requirements, 5 change	83,837 total documents; 16,590 labeled instances after cleaning; average text length ~13 words

	requests, 27 impacted. SAT-DLink-Test: 11 change requests, 33 impacted (from same 192).	
10 **Type of requirements**	Natural-language functional requirements from real system documents (mobile platform, satellite systems)	Short natural-language requirement/task summaries — issue-tracker style tickets (Enhancement, Story, Defect, etc.)
11**Type of requirement changes**	3 change-rationale categories: **Addition** (new feature added), **Deletion** (feature removed), **Modification** (something removed AND added together)	Does **not** study "changes" directly — classifies existing requirements by **Severity** (Normal/Major/Minor/Blocker/Critical/Undecided) and **Priority** (Unassigned/High/Medium/Low)
12. Partial changes studied?	Indirectly — e.g., "Addition" of one clause or small wording edit (changing Ref-D1 to Ref-D2) resembles a partial change	Not directly — but "Minor" severity is conceptually close to a small/partial change
13 **Full changes studied?**	Indirectly — e.g., "Deletion" of a whole feature (removing dual-launch configuration) resembles a full/major change. Change size/magnitude itself is never measured, only change type.	Not directly — but "Major"/"Critical"/"Blocker" severity is conceptually close to a large/full change
14.Algorithms/models used	GPT-4o, DeepSeek-R1, LLaMA3-405B, Mistral-Large, Gemini-2.0; NLI model (nli-deberta-v3-large) for filtering; Gradient Boosting (explainability); Gemini/OpenAI embeddings (similarity baselines)	Word Embedding pooling (GloVe+FastText), SBERT, DistilBERT, BERT-base, BERT-large, RoBERTa-base, RoBERTa-large — each plain-fine-tuned AND "+adapted"
15 **Features used**	Natural language text of change rationale + full requirement text; prompt "details" (role/instruction/context) as features for Gradient Boosting explainability	Raw requirement summary text only (~13 words avg); no manually engineered features, pure end-to-end deep learning

16	**Evaluation methodology**	Predicted "impact set" compared to real ground-truth impact set per change rationale; statistical significance via Fisher's Exact Test	Train/validation/test split; 5×2 cross-validation paired F-test for significance between adapted vs. non-adapted models
17	**Evaluation metrics**	Recall/Effectiveness (% truly impacted requirements found), Cost (% of requirements analyst must check), Precision, F2-score	Accuracy, F1 (macro), weighted-F1
18	**Main experimental results**	Best method (ProReFiCIA + GPT-4o) found **85.7%** of truly impacted requirements while reviewing only **3.0%** of all requirements; adding RAG raised this to **95.8%** effectiveness at 3.4% cost — beat all baselines	Adaptive fine-tuning **improved every model on every task**, no case of harm. Type easiest (~80.5% Acc / 78.7% F1 for RoBERTa-base+adapted). Priority harder (69% data in one class). **Severity hardest of all** — barely beats simple baselines.
20	**Limitations (stated)**	Small sample sizes (few change rationales) → some statistical tests borderline; SAT-DLink data private/not shareable; may not generalize to very large systems; only tested in satellite/mobile domain	Only one data source (DOORS) used; all 3 tasks suffer from severe class imbalance; general-purpose (not SE-specific) pretrained checkpoints used
21	**Explicit future work**	Investigate false positives in more depth; extend ProReFiCIA to other artifact types (test cases, use cases, source code)	Test on more/diverse datasets; use data augmentation for class imbalance; try an SE-specific pretrained Transformer
22	**What the paper does NOT address**	Never separates or measures **how large/partial/full** a change is — only change *type* (Addition/Deletion/Modification), never magnitude. Does not test if full changes are harder to trace than partial ones.	Never frames labels as "requirement change magnitude" — it's a text-classification paper, not a change-impact paper. Never tests whether Severity relates to size of underlying change. **DOORS dataset is explicitly NOT public** (proprietary) — important for any dataset-search step.

RESEARCH GAPS:

Cross-paper comparison on Partial vs Full changes:

Question	Paper A	Paper B
Does it study partial changes?	Indirectly (Addition/small wording edits	Indirectly (Minor/Normal severity
Does it study full changes?	Indirectly (Deletion of whole feature)	Indirectly (Major/Critical/Blocker severity)
Does it treat all changes equally in evaluation?	**Yes** — all change rationales are scored the same way regardless of whether they are Addition, Deletion, or Modification. No breakdown of results by change type is reported	**Yes** — severity label is the outcome, not something the model performance is broken down by "size of underlying change.
Does it fail to distinguish change	**Yes, explicitly.** Effectiveness/Cost metrics are reported per-dataset, never per-change-type.	**Yes.** The paper never measures the *textual distance* (how much content actually changed) between an original requirement and a "Major" vs "Minor" one.
Does change magnitude affect performance?	**Unknown/untested** — this is not measured.	**Unknown/untested** — but the paper does note Severity is the *hardest* task, which hints (not proven) that "how serious a change/requirement is" may be harder for models to learn than "what type it is."
Do the datasets contain data that could allow partial/full analysis?	Partially — the "Addition/Deletion/Modification" tag in the change rationale already exists, so re-analysis by change type is technically possible if someone reruns their published numbers by change-type. But this exact breakdown is not in the paper.	Yes — the Severity label already gives a magnitude scale (Blocker > Critical > Major > Normal > Minor), so it is technically reusable for this exact question. But the da
Genuine research gap	Yes	Yes

Research Gap 1 — Change magnitude is never isolated as a variable

Research Gap: No paper in this set measures whether the size of a requirement change (a small partial edit vs. a full replacement/removal) affects how well an automated method can find its impact.

Evidence: Paper A defines 3 change categories (Addition/Deletion/Modification) but reports results only pooled across all of them (Table 5, 8 in the paper — no per-category breakdown).

Paper B has a ready-made severity scale (Blocker→Undecided) but never links severity to *how much text actually changed*.

Existing Approach: Both papers treat "a change" as one uniform unit of analysis.

Missing Aspect: No experiment splits results by "small/partial change" vs "large/full change" and compares recall, precision, or classification accuracy between the two groups.

Why It Matters: If large/full changes are handled worse by automated tools (which is intuitively likely, since they touch more concepts and may break more silent dependencies), then safety-critical projects need a warning flag for "this change is large — extra manual review recommended." Right now, no evidence exists either way.

Potential Research Direction: Take the existing Addition/Deletion/Modification labels (Paper A) or Severity labels (Paper B) as a rough proxy for magnitude, and statistically test whether model performance (recall/precision/F1) differs significantly between the "small change" group and the "large change" group.

Research Gap 2 — No public dataset explicitly labels requirement changes as "partial" vs "full"

Research Gap: There is no dataset (in either paper) that was purpose-built with a partial/full change label.

Evidence: Paper A's WASP/SAT-DLink datasets use Addition/Deletion/Modification, which is about *what kind* of edit happened, not *how much* changed. Paper B's DOORS dataset uses issue-tracker Severity/Priority, which describes urgency, not the size of a text edit, and is proprietary.

Existing Approach: Researchers reuse whatever labels happen to already exist in whatever dataset they have (Type/Severity/Priority, or Addition/Deletion/Modification).

Missing Aspect: A clean, operational definition of "partial change" (e.g., <30% of requirement text altered, or only one clause/attribute changed) vs. "full change" (e.g., entire requirement replaced or removed) is missing from the literature entirely.

Why It Matters: Without an agreed definition, results from different studies cannot be compared, and "change magnitude" cannot become a standard variable in Requirements Engineering research.

Potential Research Direction: Define partial/full change using a measurable text-similarity threshold (e.g., edit distance, cosine similarity between old and new requirement text) and apply it as a new label on top of an existing public dataset.

Research Gap 3 — Severity/Priority classification is much weaker than Type classification, and the cause is not explained by text features alone

Research Gap: Paper B shows Severity classification is far harder than Type classification, but never investigates *why* using change-size or requirement-complexity features — only text content was used.

Evidence: Paper B's Table 4 — Severity F1 scores stay very low (15–29%) even with adaptation, while Type F1 reaches 73–79%.

Existing Approach: Only raw requirement text (13 words average) is used as input.

Missing Aspect: No structural/change-size features (e.g., length of change description, number of components touched, whether it's Addition/Deletion/Modification) were tried as extra input signals.

Why It Matters: If Severity is fundamentally about how big/impactful a change is, then adding a "change magnitude" feature (from Gap 1/2 above) might be exactly the missing signal that could improve this weak task.

Potential Research Direction: Combine Paper A's change-type/size signal with Paper B's classification setup — test whether adding a computed "magnitude score" as an extra feature improves Severity classification F1.

GAP COMPARISON TABLE:

Research Gap	Evidence from Papers	Related to Partial/Full Changes?	Can Be Experimentally Tested?	Dataset Requirement
Gap 1: Change magnitude never isolated	Paper A Table 5/8 (pooled results only); Paper B Table 2/4 (no magnitude feature)	Yes — directly	Yes, using existing Addition/Deletion/Modification tags or Severity labels as a proxy	Needs a dataset with either change-type tags or severity tags (Paper A's WASP is public; Paper B's DOORS is NOT public)
Gap 2: No standard partial/full definition	Absent from both papers	Yes — directly	Yes, by computing text-similarity-based magnitude scores on any dataset with before/after requirement text	Needs before/after requirement text pairs (only Paper A's WASP-style data has this structure publicly)

Gap 3: Severity is hard, cause untested	Paper B Table 4 (low Severity F1)	Partially related (Severity ≈ proxy for magnitude)	Yes, if magnitude features can be engineered	Needs the DOORS dataset (NOT public) — cannot be tested with Paper B's exact data